Table des matières

Labo 04 — Extraire d'une base ou d'une API 1
Avant de commencer 2
Étape 1 — Interroger la base de l'ERP : filigrane, plan de requête, documents de modification 5
Étape 2 — Tirer les neuf tables de l'ERP par ODBC, puis prouver qu'elles égalent la RAW 6
Étape 3 — Vérifier le filigrane de décembre contre l'extrait 10
Étape 4 — Appeler l'API d'ECCC page par page, puis la comparer à l'instantané 11
Étape 5 — Versionner le code, laisser dehors données et secrets 15
Dépannage 15

Labo 04 — Extraire d'une base ou d'une API

Labo · Cours 4/12 — Formation Data Engineering

But : recevoir la plateforme et monter la base de l'ERP ; l'interroger ; la lire par ODBC et prouver que les extraits de la `RAW` racontent la même histoire qu'elle ; vérifier le filigrane de décembre ; tirer 92 jours de la vraie API météo et les comparer à l'instantané ; garder le code dans le dépôt, jamais le mot de passe. Le pourquoi de chaque étape est dans `Theorie_04_DE.md`, section 5 et annexe A.

Trois fenêtres. Chacune reste dans son dossier du début à la fin du labo : aucun `cd` à taper après l'ouverture.

Fenêtre	Dossier	Ce qu'on y fait
1. Anaconda Prompt, `formations` actif	`Projet_Boreal_intermediaire`	le Bloc-notes, Python, Git, `duckdb -f`
2. Anaconda Prompt devenue Ubuntu	`Projet_Boreal_intermediaire\plateforme`	Docker, et `psql`, le client de la base de l'ERP
3. Anaconda Prompt, à partir de l'étape 3	`Projet_Boreal_intermediaire`	l'interface web de DuckDB (`duckdb -ui`)

La base de l'ERP tourne dans un conteneur Docker, et Docker vit dans Ubuntu : tout ce qui touche la base passe par la fenêtre 2.

Liste de contrôle :

- ☐ Plateforme reçue dans `plateforme\`, base de l'ERP démarrée et chargée : 25 483 lignes dans `VBAK`
- ☐ Base interrogée par `psql` : un filigrane de 427, un plan par l'index, un plan qui lit toute la table

© 2026 iAlexMG. Tous droits réservés.

- ☐ Mot de passe recopié dans `01_de\.env`, modèle `01_de\.env.exemple` écrit ; `extraire_erp_odbc.py` lancé : neuf tables
- ☐ `controle_erp_base.sql` : 0 écart dans les deux sens sur les neuf tables
- ☐ Filigrane de décembre égal à l'extrait, ligne pour ligne
- ☐ `extraire_meteo_api.py` lancé : 92 jours en 2 pages ; 0 écart avec l'instantané, comparé en nombres
- ☐ Code, contrôle et `.env.exemple` validés dans `01_de` ; `.env` ignoré, `git status` propre

---

Avant de commencer

Vérifier ce que le Cours 03 a laissé, et le pilote ODBC. Fenêtre 1, dans `Projet_Boreal_intermediaire` :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir 01_de\ingestion
REM Attendu : deux fichiers, charger_raw.py et deposer_corpus.py.
dir plateforme
REM Attendu : deux dossiers, entrepot et lac.
python -c "import pyodbc; print(pyodbc.drivers())"
REM Attendu : une liste qui contient 'PostgreSQL Unicode(x64)'.
```

Si `git`, `dir 01_de\ingestion` ou `dir plateforme` ne donne pas son attendu, finir d'abord le Labo 03. Si le pilote manque, l'installer par l'étape 6 du guide d'installation du parcours, `Data_Domain/00_Installation/Guide_installation.md`.

Recevoir la plateforme : ses fichiers se copient tels quels depuis `_univers/plateforme/`, le dossier voisin de `Projet_Boreal_intermediaire`. Le pourquoi de chaque geste de ce bloc est dans `Theorie_04_DE.md`, annexe A.0. Fenêtre 1 :

```
copy ..\_univers\plateforme\docker-compose.yml plateforme\
REM Attendu : 1 fichier(s) copié(s).
copy ..\_univers\plateforme\Dockerfile.airflow plateforme\
REM Attendu : 1 fichier(s) copié(s).
copy ..\_univers\plateforme\.env.exemple plateforme\.env
REM Attendu : 1 fichier(s) copié(s).
notepad plateforme\.env
REM Attendu : le .env de la plateforme ouvert dans le Bloc-notes.
```


© 2026 iAlexMG. Tous droits réservés.

Choisir le mot de passe de la base de l'ERP, avant de la démarrer : elle garde celui de son premier démarrage, et un changement fait après coup ne l'atteint plus. C'est le seul endroit où vous le tapez : l'étape 2 le recopie pour le code. Sur la ligne `ERP_MOT_DE_PASSE=remplacez-moi`, remplacer `remplacez-moi` par un mot de passe de votre choix, en lettres et en chiffres seulement. Enregistrer, fermer.

Ouvrir la fenêtre 2 : une nouvelle Anaconda Prompt, dans `Projet_Boreal_intermediaire`, puis :

```
cd plateforme
wsl
```

Attendu : une invite qui finit par `…/Projet_Boreal_intermediaire/plateforme$`. Cette fenêtre est désormais Ubuntu, dans le dossier de la plateforme, et y reste. La garder ouverte jusqu'à la fin de l'étape 2 : Ubuntu, et la base avec elle, s'arrête peu après la fermeture de son dernier terminal.

Démarrer la base de l'ERP. Elle tourne dans un conteneur : un petit système isolé qui contient PostgreSQL et ses données. Le fichier `docker-compose.yml` décrit ce conteneur dans son service `erp`. Voici ce passage du fichier, à lire seulement, rien à coller :

```
erp:
  image: postgres:18
  container_name: boreal_erp
  environment:
    POSTGRES_DB: erp_boreal
    POSTGRES_PASSWORD: ${ERP_MOT_DE_PASSE}
  ports:
    - "5433:5432"
  volumes:
    - erp_donnees:/var/lib/postgresql
    - ../sources/erp/postgresql:/erp:ro
```

Ligne	Ce qu'elle déclare	Où le labo la retrouve
`erp:`	le service : le nom que les commandes `docker compose` désignent	`up -d erp`, `exec … erp`
`image: postgres:18`	le logiciel du conteneur : l'image officielle de PostgreSQL 18, téléchargée au premier lancement	l'attendu de `up`
`container_name: boreal_erp`	le nom du conteneur que le service démarre	les attendus de `up` et de `ps`
`POSTGRES_DB: erp_boreal`	la base de données, créée au premier démarrage	`psql -d erp_boreal`, le script de l'étape 2
`POSTGRES_PASSWORD`	le mot de passe, lu dans `plateforme\.env` au premier démarrage seulement	l'étape 2

© 2026 iAlexMG. Tous droits réservés.

Ligne	Ce qu'elle déclare	Où le labo la retrouve
`"5433:5432"`	le port : PostgreSQL écoute sur le 5432 dans le conteneur, Windows le joint sur le 5433	l'attendu de `ps`, le script de l'étape 2
`erp_donnees:/var/lib/postgresql`	le volume : les fichiers de la base, qui survivent à l'arrêt du conteneur	nulle part : Docker s'en occupe
`../sources/erp/postgresql:/erp:ro`	le dossier `sources\erp\postgresql` du projet, visible dans le conteneur sous le nom `/erp`, en lecture seule (`ro`)	`-w /erp`, au chargement

Le dossier `sources\erp\` du projet n'est donc pas le service `erp` : c'est là que l'ERP dépose ses livraisons, les extraits CSV des Cours 01 à 03 dans `extraits\`, et la copie de sa base dans `postgresql\`. La dernière ligne du passage est le seul lien entre les deux. Fenêtre 2 :

```
docker compose up -d erp
# Attendu : au premier lancement, le téléchargement de l'image postgres:18,
# puis Container boreal_erp Started.
docker compose ps
# Attendu : boreal_erp, Up, 0.0.0.0:5433->5432/tcp.
```

Morceau	Ce qu'il fait
`docker compose`	lit `docker-compose.yml`, et le `.env` voisin, dans le dossier courant
`up -d erp`	crée et démarre le seul service `erp`, en arrière-plan (`-d`) : les autres services du fichier attendent leur cours
`ps`	liste les conteneurs du fichier, leur état et leurs ports

PostgreSQL prépare sa base pendant quelques secondes après le démarrage : compter jusqu'à dix avant la suite.

Charger la base avec le script que l'ERP a livré, `charger_erp.sql`, rangé avec ses CSV dans `sources\erp\postgresql`, que le conteneur voit sous `/erp`. Fenêtre 2 :

```
docker compose exec -w /erp erp psql -U postgres -d erp_boreal -f charger_erp.sql
# Attendu : NOTICE:  schema "sap" does not exist, skipping,
# puis un tableau de 9 lignes, de CDHDR (53) à ZSERIE (27775), dont VBAK 25483.
```

Morceau	Ce qu'il fait
`docker compose exec`	lance une commande dans le conteneur d'un service déjà démarré

© 2026 iAlexMG. Tous droits réservés.

Morceau	Ce qu'il fait
`-w /erp`	le dossier où la lancer, dans le conteneur : `/erp`, soit `sources\erp\postgresql` vu de l'intérieur
`erp`	le service visé, donc son conteneur, `boreal_erp`
`psql`	le client de PostgreSQL, installé dans le conteneur
`-U postgres -d erp_boreal`	l'utilisateur et la base
`-f charger_erp.sql`	le script à exécuter, trouvé dans `/erp`

Les deux `erp` de la commande ne se confondent pas : `/erp`, avec sa barre oblique, est un dossier ; `erp`, seul, est le service.

Étape 1 — Interroger la base de l'ERP : filigrane, plan de requête, documents de modification

Théorie : `Theorie_04_DE.md`, annexe A.1.

Cette étape se fait dans la fenêtre 2, Ubuntu : `psql` s'ouvre dans le conteneur de la base, par une commande Docker. Aucun client PostgreSQL à installer sous Windows.

1.1 Ouvrir `psql` dans le conteneur. Fenêtre 2 :

```
docker compose exec erp psql -U postgres -d erp_boreal
```

Attendu : l'invite `erp_boreal=#`. Ce bloc n'a pas d'attendu en commentaire : collée après la commande, la ligne partirait dans `psql`. Le SQL de cette étape se tape à cette invite, ni dans DuckDB ni dans Ubuntu : coller chaque requête, commentaire compris, jusqu'à son `;`.

1.2 Compter le filigrane de décembre 2025 : les commandes créées ou modifiées depuis le 1er décembre.

```
-- Attendu : 427.
SELECT COUNT(*) AS decembre
FROM sap.vbak
WHERE erdat >= '20251201' OR aedat >= '20251201';
```

1.3 Voir le plan d'un filtre sur la seule date de modification.

```
-- Attendu : Index Scan using ix_vbak_aedat on vbak. Dans la minute qui suit le chargement :
-- Bitmap Index Scan on ix_vbak_aedat. Les deux passent par l'index.
EXPLAIN SELECT * FROM sap.vbak WHERE aedat >= '20251201';
```

1.4 Voir le plan du filigrane complet.

© 2026 iAlexMG. Tous droits réservés.

```
-- Attendu : Seq Scan on vbak.
EXPLAIN SELECT * FROM sap.vbak WHERE erdat >= '20251201' OR aedat >= '20251201';
```

1.5 Lire les documents de modification : qui a changé quoi, et quand.

```
-- Attendu : 5 lignes. 20221117 XD02 LOEVM 1, 20230717 XD02 LOEVM 1, 20240701 ZPALIER KUKLA 49,
-- 20241213 XD02 LOEVM 1, 20250410 XD02 LOEVM 1.
SELECT h.udate, h.tcode, p.fname, COUNT(*) AS changements
FROM sap.cdhdr h
JOIN sap.cdpos p USING (mandt, objectclas, objectid, changenr)
GROUP BY h.udate, h.tcode, p.fname
ORDER BY h.udate;
```

1.6 Quitter `psql` par `\q`. Laisser la fenêtre 2 ouverte.

Étape 2 — Tirer les neuf tables de l'ERP par ODBC, puis prouver qu'elles égalent la RAW

Théorie : `Theorie_04_DE.md`, annexe A.2.

2.1 Donner le mot de passe au code, sans le retaper : recopier sa ligne du `.env` de la plateforme dans le `.env` du dépôt. Fenêtre 1 :

```
findstr /B "ERP_MOT_DE_PASSE=" plateforme\.env > 01_de\.env
type 01_de\.env
REM Attendu : une seule ligne, ERP_MOT_DE_PASSE= suivi de votre mot de passe.
```

Le mot de passe vit maintenant à deux endroits, et un modèle sans secret l'accompagne :

Fichier	Qui le lit	Ce qu'il contient
`plateforme\.env`	`docker compose`, qui a créé la base	le mot de passe, tapé une fois, avant de commencer
`01_de\.env`	le script d'extraction, qui se connecte à la base	la même ligne, recopiée en 2.1 ; Git l'ignore
`01_de\.env.exemple`	personne : c'est le modèle, versionné	`remplacez-moi`, jamais le vrai mot de passe

2.2 Écrire le modèle du `.env`, sans secret : c'est lui qui ira dans le dépôt. Fenêtre 1 :

```
notepad 01_de\.env.exemple
```

Accepter la création. Coller, enregistrer, fermer :

© 2026 iAlexMG. Tous droits réservés.

```
# Modèle du .env : copiez-le en .env et remplacez la valeur par le mot de passe de la base de l'ERP,
# celui du .env de la plateforme. Ce modèle va dans Git, le .env jamais.
ERP_MOT_DE_PASSE=remplacez-moi
```

2.3 Créer le script d'extraction. Fenêtre 1 :

```
notepad 01_de\ingestion\extraire_erp_odbc.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""extraire_erp_odbc.py : les neuf tables de la base de l'ERP, lues par ODBC, déposées dans le bronze.

Lancement, depuis Projet_Boreal_intermediaire/01_de, le conteneur démarré : python ingestion/extraire_erp_odbc.py
"""
import csv
import sys
import time
from pathlib import Path

import duckdb
import pyodbc

PROJET = Path(__file__).resolve().parents[2]
TABLES = ["t005u", "kna1", "mara", "makt", "vbak", "vbap", "cdhdr", "cdpos", "zserie"]
PAQUET = 10_000


def lire_env(cle: str) -> str:
    """Une valeur de 01_de/.env, un fichier de lignes CLE=valeur que Git ignore."""
    env = PROJET / "01_de" / ".env"
    if env.exists():
        for ligne in env.read_text(encoding="utf-8").splitlines():
            nom, _, valeur = ligne.partition("=")
            if nom.strip() == cle and not ligne.lstrip().startswith("#"):
                return valeur.strip()
    sys.exit(f"{cle} introuvable dans {env} : copiez .env.exemple en .env et remplissez-le.")


def main():
    pilotes = [p for p in pyodbc.drivers() if p.startswith("PostgreSQL Unicode")]
    if not pilotes:
        sys.exit(f"Pilote psqlODBC introuvable. Pilotes ODBC installés : {pyodbc.drivers()}")
```


© 2026 iAlexMG. Tous droits réservés.

```
    chaine = (f"DRIVER={{{pilotes[0]}}};SERVER=localhost;PORT=5433;DATABASE=erp_boreal;"
              f"UID=postgres;PWD={lire_env('ERP_MOT_DE_PASSE')}")
    sortie = PROJET / "plateforme" / "lac" / "bronze" / "erp_base"
    sortie.mkdir(parents=True, exist_ok=True)

    debut = time.perf_counter()
    connexion = pyodbc.connect(chaine)
    curseur = connexion.cursor()
    duck = duckdb.connect()
    for table in TABLES:
        curseur.execute(f"SELECT * FROM sap.{table}")
        colonnes = [c[0].upper() for c in curseur.description]
        temporaire = sortie / f"{table}.csv"
        lignes = 0
        with open(temporaire, "w", newline="", encoding="utf-8") as fichier:
            ecrivain = csv.writer(fichier)
            ecrivain.writerow(colonnes)
            while paquet := curseur.fetchmany(PAQUET):
                ecrivain.writerows(paquet)
                lignes += len(paquet)
        # Le bronze garde le texte reçu ; DuckDB le range en Parquet compressé.
        duck.execute(f"""
            COPY (SELECT * FROM read_csv('{temporaire.as_posix()}', header = true, all_varchar = true))
            TO '{(sortie / f"{table}.parquet").as_posix()}' (FORMAT parquet, COMPRESSION zstd)""")
        temporaire.unlink()
        print(f"sap.{table:7} {lignes:>7} lignes")
    connexion.close()
    print(f"Neuf tables extraites en {time.perf_counter() - debut:.1f} s : "
          f"{sortie.relative_to(PROJET).as_posix()}")


if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    main()
```

2.4 Lancer l'extraction. La fenêtre 2 doit être ouverte : la base vit dans son conteneur. Fenêtre 1 :

```
python 01_de\ingestion\extraire_erp_odbc.py
REM Attendu : neuf lignes, de sap.t005u 20 lignes à sap.zserie 27775 lignes,
REM puis Neuf tables extraites en ... s : plateforme/lac/bronze/erp_base (environ 5 s chez l'auteur).
```

2.5 Créer le script de contrôle. Fenêtre 1 :

© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\sql\controle_erp_base.sql
```

Accepter la création. Coller, enregistrer, fermer :

```
-- 01_de/sql/controle_erp_base.sql : les extraits de la RAW retrouvent-ils la base de l'ERP ?
-- Lancement, depuis Projet_Boreal_intermediaire, la ligne de commande DuckDB fermée : duckdb -f 01_de/sql/controle_erp_base.sql

ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
USE entrepot;

WITH
-- La dernière version de chaque clé, colonnes de lignage retirées. Les clés sont celles du dictionnaire.
t005u AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_t005u
          QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, SPRAS, LAND1, BLAND ORDER BY _lot DESC) = 1),
kna1 AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_kna1
         QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, KUNNR ORDER BY _lot DESC) = 1),
mara AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_mara
         QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, MATNR ORDER BY _lot DESC) = 1),
makt AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_makt
         QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, MATNR, SPRAS ORDER BY _lot DESC) = 1),
vbak AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_vbak
         QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN ORDER BY _lot DESC) = 1),
vbap AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_vbap
         QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN, POSNR ORDER BY _lot DESC) = 1),
cdhdr AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_cdhdr
          QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, OBJECTCLAS, OBJECTID, CHANGENR ORDER BY _lot DESC) = 1),
cdpos AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_cdpos
          QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, OBJECTCLAS, OBJECTID, CHANGENR, TABNAME, TABKEY, FNAME, CHNGIND ORDER BY
_lot DESC) = 1),
zserie AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM raw.erp_zserie
           QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, SERNR ORDER BY _lot DESC) = 1),
-- La même table, lue dans la base par ODBC.
b_t005u AS (FROM 'plateforme/lac/bronze/erp_base/t005u.parquet'),
b_kna1 AS (FROM 'plateforme/lac/bronze/erp_base/kna1.parquet'),
b_mara AS (FROM 'plateforme/lac/bronze/erp_base/mara.parquet'),
b_makt AS (FROM 'plateforme/lac/bronze/erp_base/makt.parquet'),
b_vbak AS (FROM 'plateforme/lac/bronze/erp_base/vbak.parquet'),
b_vbap AS (FROM 'plateforme/lac/bronze/erp_base/vbap.parquet'),
b_cdhdr AS (FROM 'plateforme/lac/bronze/erp_base/cdhdr.parquet'),
b_cdpos AS (FROM 'plateforme/lac/bronze/erp_base/cdpos.parquet'),
b_zserie AS (FROM 'plateforme/lac/bronze/erp_base/zserie.parquet')
SELECT 'T005U' AS table_sap, (SELECT COUNT(*) FROM t005u) AS lignes_raw, (SELECT COUNT(*) FROM b_t005u) AS lignes_base,
       (SELECT COUNT(*) FROM (FROM t005u EXCEPT FROM b_t005u)) AS raw_sans_base,
```


© 2026 iAlexMG. Tous droits réservés.

```
       (SELECT COUNT(*) FROM (FROM b_t005u EXCEPT FROM t005u)) AS base_sans_raw
UNION ALL
SELECT 'KNA1', (SELECT COUNT(*) FROM kna1), (SELECT COUNT(*) FROM b_kna1),
       (SELECT COUNT(*) FROM (FROM kna1 EXCEPT FROM b_kna1)), (SELECT COUNT(*) FROM (FROM b_kna1 EXCEPT FROM kna1))
UNION ALL
SELECT 'MARA', (SELECT COUNT(*) FROM mara), (SELECT COUNT(*) FROM b_mara),
       (SELECT COUNT(*) FROM (FROM mara EXCEPT FROM b_mara)), (SELECT COUNT(*) FROM (FROM b_mara EXCEPT FROM mara))
UNION ALL
SELECT 'MAKT', (SELECT COUNT(*) FROM makt), (SELECT COUNT(*) FROM b_makt),
       (SELECT COUNT(*) FROM (FROM makt EXCEPT FROM b_makt)), (SELECT COUNT(*) FROM (FROM b_makt EXCEPT FROM makt))
UNION ALL
SELECT 'VBAK', (SELECT COUNT(*) FROM vbak), (SELECT COUNT(*) FROM b_vbak),
       (SELECT COUNT(*) FROM (FROM vbak EXCEPT FROM b_vbak)), (SELECT COUNT(*) FROM (FROM b_vbak EXCEPT FROM vbak))
UNION ALL
SELECT 'VBAP', (SELECT COUNT(*) FROM vbap), (SELECT COUNT(*) FROM b_vbap),
       (SELECT COUNT(*) FROM (FROM vbap EXCEPT FROM b_vbap)), (SELECT COUNT(*) FROM (FROM b_vbap EXCEPT FROM vbap))
UNION ALL
SELECT 'CDHDR', (SELECT COUNT(*) FROM cdhdr), (SELECT COUNT(*) FROM b_cdhdr),
       (SELECT COUNT(*) FROM (FROM cdhdr EXCEPT FROM b_cdhdr)), (SELECT COUNT(*) FROM (FROM b_cdhdr EXCEPT FROM cdhdr))
UNION ALL
SELECT 'CDPOS', (SELECT COUNT(*) FROM cdpos), (SELECT COUNT(*) FROM b_cdpos),
       (SELECT COUNT(*) FROM (FROM cdpos EXCEPT FROM b_cdpos)), (SELECT COUNT(*) FROM (FROM b_cdpos EXCEPT FROM cdpos))
UNION ALL
SELECT 'ZSERIE', (SELECT COUNT(*) FROM zserie), (SELECT COUNT(*) FROM b_zserie),
       (SELECT COUNT(*) FROM (FROM zserie EXCEPT FROM b_zserie)), (SELECT COUNT(*) FROM (FROM b_zserie EXCEPT FROM zserie));
```

2.6 Lancer le contrôle. Il ouvre l'entrepôt : aucune interface DuckDB ne doit le tenir. Fenêtre 1 :

```
duckdb -f 01_de/sql/controle_erp_base.sql
REM Attendu : 9 lignes, de T005U (20 et 20) à ZSERIE (27775 et 27775),
REM et 0 partout dans raw_sans_base et base_sans_raw.
```

La suite du labo n'a plus besoin de la base de l'ERP ; les exercices, si. La fenêtre 2 peut se fermer : à sa prochaine ouverture, la base repart d'elle-même, avec ses données.

Étape 3 — Vérifier le filigrane de décembre contre l'extrait

Théorie : `Theorie_04_DE.md`, annexe A.3.

Ouvrir la fenêtre 3 : une nouvelle Anaconda Prompt, dans `Projet_Boreal_intermediaire`, puis :

```
conda activate formations
duckdb -ui
```


© 2026 iAlexMG. Tous droits réservés.

Laisser cette fenêtre ouverte : la fermer arrête l'interface. Créer un notebook neuf (+ à côté de Notebooks) et le renommer « Labo 04 » (double-clic sur son titre, puis Entrée). Une requête par cellule, Ctrl+Entrée pour l'exécuter.

3.1 Attacher l'entrepôt, en lecture seule.

```
-- Attendu : 0 rows returned.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
```

3.2 Compter les en-têtes de l'extrait de décembre.

```
-- Attendu : 427.
SELECT COUNT(*) AS en_tetes
FROM entrepot.raw.erp_vbak
WHERE _lot = '2025-12';
```

3.3 Découper le filigrane, sur l'état de la base au 2025-12-31 : ce qu'un filtre sur la seule date de modification raterait.

```
-- Attendu : 427, 37, 390 et 9.
WITH etat AS (
    SELECT * FROM entrepot.raw.erp_vbak
    QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN ORDER BY _lot DESC) = 1)
SELECT COUNT(*) FILTER (WHERE ERDAT >= '20251201' OR AEDAT >= '20251201') AS filigrane,
       COUNT(*) FILTER (WHERE AEDAT >= '20251201') AS modifiees_en_decembre,
       COUNT(*) FILTER (WHERE ERDAT >= '20251201' AND AEDAT = '00000000') AS creees_jamais_modifiees,
       COUNT(*) FILTER (WHERE ERDAT < '20251201' AND AEDAT >= '20251201') AS anciennes_modifiees
FROM etat;
```

3.4 Prouver que le filigrane rend les mêmes lignes que l'extrait de décembre.

```
-- Attendu : 0 et 0.
WITH etat AS (
    SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM entrepot.raw.erp_vbak
    QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN ORDER BY _lot DESC) = 1),
filigrane AS (SELECT * FROM etat WHERE ERDAT >= '20251201' OR AEDAT >= '20251201'),
extrait AS (SELECT * EXCLUDE (_fichier, _lot, _charge_le) FROM entrepot.raw.erp_vbak WHERE _lot = '2025-12')
SELECT (SELECT COUNT(*) FROM (FROM filigrane EXCEPT FROM extrait)) AS filigrane_sans_extrait,
       (SELECT COUNT(*) FROM (FROM extrait EXCEPT FROM filigrane)) AS extrait_sans_filigrane;
```

Étape 4 — Appeler l'API d'ECCC page par page, puis la comparer à l'instantané

Théorie : `Theorie_04_DE.md`, annexe A.4.

4.1 Créer le script. Fenêtre 1 :

© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\ingestion\extraire_meteo_api.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""extraire_meteo_api.py : la vraie API d'ECCC, page par page, déposée dans le bronze.

Lancement, depuis Projet_Boreal_intermediaire/01_de : python ingestion/extraire_meteo_api.py
"""
import json
import sys
import time
import urllib.error
import urllib.parse
import urllib.request
from pathlib import Path

PROJET = Path(__file__).resolve().parents[2]
API = "https://api.weather.gc.ca/collections/climate-daily/items"
STATION, DEBUT, FIN = "7060400", "2025-10-01", "2025-12-31"
PAGE = 50
ESSAIS = 4


def lire_page(url: str) -> dict:
    """Une page de l'API. Réessaie après une coupure, un 429 ou une erreur 5xx."""
    for essai in range(1, ESSAIS + 1):
        try:
            with urllib.request.urlopen(url, timeout=60) as reponse:
                return json.load(reponse)
        except urllib.error.HTTPError as e:
            if e.code != 429 and e.code < 500:
                raise                                   # requête fausse : inutile de réessayer
            erreur = f"HTTP {e.code}"
        except urllib.error.URLError as e:
            erreur = str(e.reason)
        if essai == ESSAIS:
            raise RuntimeError(f"abandon après {ESSAIS} essais ({erreur}) : {url}")
        attente = 2 ** essai                            # 2, 4, puis 8 secondes
        print(f"  essai {essai} raté ({erreur}), nouvel essai dans {attente} s")
        time.sleep(attente)
```


© 2026 iAlexMG. Tous droits réservés.

```
def main():
    parametres = {"f": "json", "CLIMATE_IDENTIFIER": STATION, "sortby": "LOCAL_DATE",
                  "datetime": f"{DEBUT} 00:00:00/{FIN} 00:00:00", "limit": PAGE}
    url = f"{API}?{urllib.parse.urlencode(parametres)}"
    sortie = PROJET / "plateforme" / "lac" / "bronze" / "meteo_api" / f"quotidien_{STATION}_{DEBUT}_{FIN}.jsonl"
    sortie.parent.mkdir(parents=True, exist_ok=True)

    pages = recus = 0
    annonces = None
    with open(sortie, "w", encoding="utf-8", newline="\n") as fichier:
        while url:
            page = lire_page(url)
            pages += 1
            annonces = page["numberMatched"]
            for entite in page["features"]:
                fichier.write(json.dumps(entite["properties"], ensure_ascii=False) + "\n")
            recus += page["numberReturned"]
            print(f"  page {pages} : {page['numberReturned']} jours")
            # La page suivante, si l'API en annonce une.
            url = next((lien["href"] for lien in page["links"] if lien["rel"] == "next"), None)

    print(f"{recus} jours reçus sur {annonces} annoncés, en {pages} pages : "
          f"{sortie.relative_to(PROJET).as_posix()}")
    if recus != annonces:
        sys.exit("Écart entre les jours reçus et les jours annoncés : l'extraction est incomplète.")


if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    main()
```

4.2 Lancer l'extraction. Il faut Internet : c'est la vraie API d'Environnement et Changement climatique Canada. Fenêtre 1 :

```
python 01_de\ingestion\extraire_meteo_api.py
REM Attendu : page 1 : 50 jours, page 2 : 42 jours, puis
REM 92 jours reçus sur 92 annoncés, en 2 pages : plateforme/lac/bronze/meteo_api/...
```

4.3 Relancer l'extraction, pour prouver qu'elle se rejoue sans rien doubler. Fenêtre 1 :

```
python 01_de\ingestion\extraire_meteo_api.py
REM Attendu : la même sortie. Le script réécrit le même fichier, qui garde ses 92 jours.
```

4.4 Voir les types que DuckDB lit dans le JSON de l'API. Fenêtre 3, dans le notebook :

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 5 lignes. LOCAL_DATE TIMESTAMP, MEAN_TEMPERATURE DOUBLE, TOTAL_SNOW DOUBLE,
-- SNOW_ON_GROUND BIGINT, CLIMATE_IDENTIFIER VARCHAR.
DESCRIBE SELECT LOCAL_DATE, MEAN_TEMPERATURE, TOTAL_SNOW, SNOW_ON_GROUND, CLIMATE_IDENTIFIER
FROM read_json('plateforme/lac/bronze/meteo_api/*.jsonl');
```

4.5 Faux écarts voulus : comparer l'API à l'instantané de la `RAW` comme du texte.

```sql
-- Attendu : 92 jours, 5 écarts de température moyenne, 45 écarts de précipitations.
WITH api AS (SELECT * FROM read_json('plateforme/lac/bronze/meteo_api/*.jsonl')),
instantane AS (
    SELECT * FROM entrepot.raw.meteo_quotidien
    WHERE CLIMATE_IDENTIFIER = '7060400' AND LOCAL_DATE BETWEEN '2025-10-01' AND '2025-12-31')
SELECT COUNT(*) AS jours,
       COUNT(*) FILTER (WHERE CAST(a.MEAN_TEMPERATURE AS VARCHAR) IS DISTINCT FROM i.MEAN_TEMPERATURE) AS temperature,
       COUNT(*) FILTER (WHERE CAST(a.TOTAL_PRECIPITATION AS VARCHAR) IS DISTINCT FROM i.TOTAL_PRECIPITATION) AS precipitations
FROM api a
JOIN instantane i ON i.LOCAL_DATE = CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR);
```

4.6 Voir les cinq écarts de température.

```sql
-- Attendu : 5 lignes. 2025-10-08, 6.0 contre 6 ; 2025-10-09, 3.0 contre 3 ; 2025-10-25, 6.0 contre 6 ;
-- 2025-11-11, -2.0 contre -2 ; 2025-12-07, -19.0 contre -19.
SELECT CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR) AS jour, a.MEAN_TEMPERATURE AS api, i.MEAN_TEMPERATURE AS instantane
FROM read_json('plateforme/lac/bronze/meteo_api/*.jsonl') a
JOIN entrepot.raw.meteo_quotidien i
  ON i.CLIMATE_IDENTIFIER = '7060400' AND i.LOCAL_DATE = CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR)
WHERE CAST(a.MEAN_TEMPERATURE AS VARCHAR) IS DISTINCT FROM i.MEAN_TEMPERATURE
ORDER BY jour;
```

4.7 Refaire la comparaison dans le bon type : en nombres.

```sql
-- Attendu : 92 jours, 0 et 0.
WITH api AS (SELECT * FROM read_json('plateforme/lac/bronze/meteo_api/*.jsonl')),
instantane AS (
    SELECT * FROM entrepot.raw.meteo_quotidien
    WHERE CLIMATE_IDENTIFIER = '7060400' AND LOCAL_DATE BETWEEN '2025-10-01' AND '2025-12-31')
SELECT COUNT(*) AS jours,
       COUNT(*) FILTER (WHERE a.MEAN_TEMPERATURE IS DISTINCT FROM CAST(i.MEAN_TEMPERATURE AS DOUBLE)) AS temperature,
       COUNT(*) FILTER (WHERE a.TOTAL_PRECIPITATION IS DISTINCT FROM CAST(i.TOTAL_PRECIPITATION AS DOUBLE)) AS precipitations
FROM api a
JOIN instantane i ON i.LOCAL_DATE = CAST(CAST(a.LOCAL_DATE AS DATE) AS VARCHAR);
```


© 2026 iAlexMG. Tous droits réservés.

À retenir — deux valeurs se comparent dans le type de ce qu'elles représentent. `6.0` et `6` sont deux textes différents et un seul nombre : comparées en texte, l'API et l'instantané semblaient différer 50 fois ; en nombres, pas une seule. On provoque ces faux écarts pour apprendre à s'en méfier : une comparaison dans le mauvais type invente des écarts, ou en cache de vrais.

4.8 Détacher l'entrepôt.

```
-- Attendu : 0 rows returned.
DETACH entrepot;
```

Étape 5 — Versionner le code, laisser dehors données et secrets

Théorie : `Theorie_04_DE.md`, annexe A.5.

Fenêtre 1. `git -C 01_de` lance Git dans le dépôt sans quitter `Projet_Boreal_intermediaire`, et les chemins qui suivent partent de `01_de` :

```
git -C 01_de add ingestion\extraire_erp_odbc.py ingestion\extraire_meteo_api.py sql\controle_erp_base.sql .env.exemple
REM Attendu : rien. Les quatre fichiers sont prêts à être validés.
git -C 01_de check-ignore -v .env
REM Attendu : .gitignore:2:.env suivi de .env. Le .env est ignoré, par la deuxième ligne du .gitignore.
git -C 01_de commit -m "Cours 04 : base de l'ERP par ODBC et API météo"
REM Attendu : 4 files changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Pour tout fermer : `.exit` à l'invite `D` de la fenêtre 3, puis fermer l'onglet du navigateur. La fenêtre 2 peut se fermer aussi.

Dépannage

Symptôme	Correctif
`SELECT: command not found`, ou `syntax error near unexpected token`	du SQL collé dans Ubuntu au lieu de `psql` : ouvrir `psql` (1.1), puis recoller ; si un fichier `=` est apparu dans `plateforme` (le `>=` pris pour une redirection), l'effacer par `rm ./=`
`REM: command not found` dans Ubuntu	une ligne `REM` collée dans la fenêtre 2 : sans effet, continuer ; dans Ubuntu, les commentaires commencent par `#`
`no configuration file provided: not found`	`docker compose` lancé ailleurs que dans `plateforme` : fermer la fenêtre 2 et la rouvrir comme au début

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Correctif
`The "ERP_MOT_DE_PASSE" variable is not set. Defaulting to a blank string.`	`plateforme\.env` manque, ou sa ligne est mal écrite : le refaire, puis `docker compose down -v`, `docker compose up -d erp` et `charger_erp.sql`, pour réinitialiser la base avec le mot de passe
`connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: No such file or directory`, ou `the database system is starting up`	PostgreSQL prépare encore sa base : attendre quelques secondes, puis relancer la même commande
`password authentication failed for user "postgres"` à l'étape 2	le mot de passe a changé dans `plateforme\.env` après le premier démarrage de la base, qui garde l'ancien : remettre l'ancien dans `plateforme\.env` et refaire 2.1 ; ou aligner la base, dans la fenêtre 2, par `docker compose exec erp psql -U postgres -d erp_boreal -c "ALTER USER postgres PASSWORD 'votre-valeur';"`
`Bind for 0.0.0.0:5433 failed: port is already allocated`	un autre conteneur publie déjà le port 5433, par exemple dans Docker Desktop : l'arrêter (guide d'installation, étape 4)
`pyodbc.OperationalError: … port 5433 failed: Connection refused` à l'étape 2	le conteneur est arrêté, Ubuntu s'étant arrêtée avec son dernier terminal : rouvrir la fenêtre 2, `docker compose ps`, au besoin `docker compose up -d erp`
`Pilote psqlODBC introuvable. Pilotes ODBC installés : […]`	psqlODBC manque, ou c'est la version 32 bits : reprendre l'étape 6 du guide d'installation
`ModuleNotFoundError: No module named 'pyodbc'`	l'environnement `formations` n'est pas actif : `conda activate formations`, puis relancer
`ERP_MOT_DE_PASSE introuvable dans …\01_de\.env`	le `.env` du dépôt manque ou est vide : refaire 2.1 depuis `Projet_Boreal_intermediaire`
`python: can't open file` sur un script de `01_de\ingestion`	la fenêtre 1 n'est pas dans `Projet_Boreal_intermediaire`, ou le Bloc-notes a enregistré le script en `.py.txt` (`dir 01_de\ingestion`) : revenir dans `Projet_Boreal_intermediaire`, ou `ren` pour retirer le `.txt`
`L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez.`, ou un `IO Error` de verrou au contrôle de 2.6	l'interface tient l'entrepôt : `DETACH entrepot;` dans le notebook, ou `.exit` dans la fenêtre 3, puis relancer
`controle_erp_base.sql` rend des écarts	la `RAW` ou le bronze `erp_base` est incomplet : relancer `charger_raw.py`, puis `extraire_erp_odbc.py`, puis le contrôle

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Correctif
`Binder Error: Catalog "entrepot" does not exist!`	l'entrepôt n'est pas attaché : relancer la cellule de 3.1
`extraire_meteo_api.py` affiche `essai 1 raté`, puis `abandon après 4 essais`	pas d'accès à Internet, ou l'API d'ECCC est indisponible : vérifier la connexion, puis relancer plus tard ; le script réécrit le même fichier
`git status` montre `.env` parmi les fichiers à valider	le `.gitignore` du Cours 01 manque sa ligne `.env` : la remettre (Labo 01, étape 1) avant tout commit

© 2026 iAlexMG. Tous droits réservés.